STATE MUTATION

CFD: 90%

DIAGNOSIS

PDF Export

← New Analysis

OBSERVED SYMPTOM

COPY

After navigating between users, the dashboard briefly shows the new user's data then reverts to the previous user's data. Bug only happens on second navigation onwards — first load is always fine.

REPRODUCTION PATH

1. 1. Load the `SessionDashboard` component with `userId='userA'`.
2. 2. Observe `userA`'s data loading and displaying correctly.
3. 3. Navigate to a different `userId`, e.g., `userId='userB'` (by changing the prop passed to `SessionDashboard`).
4. 4. Observe that `userB`'s data loads and displays briefly.
5. 5. Navigate back to `userId='userA'` (the second navigation).
6. 6. Observe that `userA`'s data loads and displays briefly, then the dashboard reverts to `userB`'s data (or potentially `userA`'s old data if a race occurs differently).

CONFIDENCE EVIDENCE

COPY

VERIFIED:

- bug9claude/sessionStore.js L1: `let _activeRef = { current: true };` - `_activeRef` is a global mutable variable.
- bug9claude/sessionStore.js L8: `_activeRef = { current: true };` - `resetSession` reassigns the global `_activeRef` variable to a new object.
- bug9claude/sessionStore.js L12: `_activeRef.current = false;` - `deactivateSession` mutates the `.current` property of the object *currently referenced by the global `_activeRef` variable*.

- bug9claude/useSessionData.js L9: `resetSession();` - Called in `useEffect` body.
- bug9claude/useSessionData.js L10: `const activeRef = getActiveRef();` - Captures the *current* global `_activeRef` object for the closure.
- bug9claude/useSessionData.js L32: `deactivateSession();` - Called in `useEffect` cleanup. This means `deactivateSession` operates on the global `_activeRef` *after* the new `useEffect` might have reassigned it (or more precisely, it might unintentionally deactivate a new session's flag if timings are tight, or fail to deactivate the old one correctly).

CONCEPT TO LEARN

COPY

SHARED MUTABLE STATE

Temporal Coupling in Global Mutable Variables

This pattern creates subtle race conditions and unpredictable behavior. When a global `let` variable holds a reference to an object, and that variable is sometimes reassigned to a *new object* (like in `resetSession`) and other times has properties of its *currently referenced object* mutated (like in `deactivateSession`), it becomes extremely difficult to reason about which object instance any given function is operating on, especially in asynchronous or lifecycle-driven environments. This leads to stale data, incorrect UI states, or missed updates because the 'active' flag isn't tracking the correct, isolated session.

> **PATTERN TO AVOID:**
> Using a global 'let' variable that is both reassigned to new objects and has its properties mutated by different functions in a lifecycle-dependent manner, especially when those functions are invoked from different component instances.

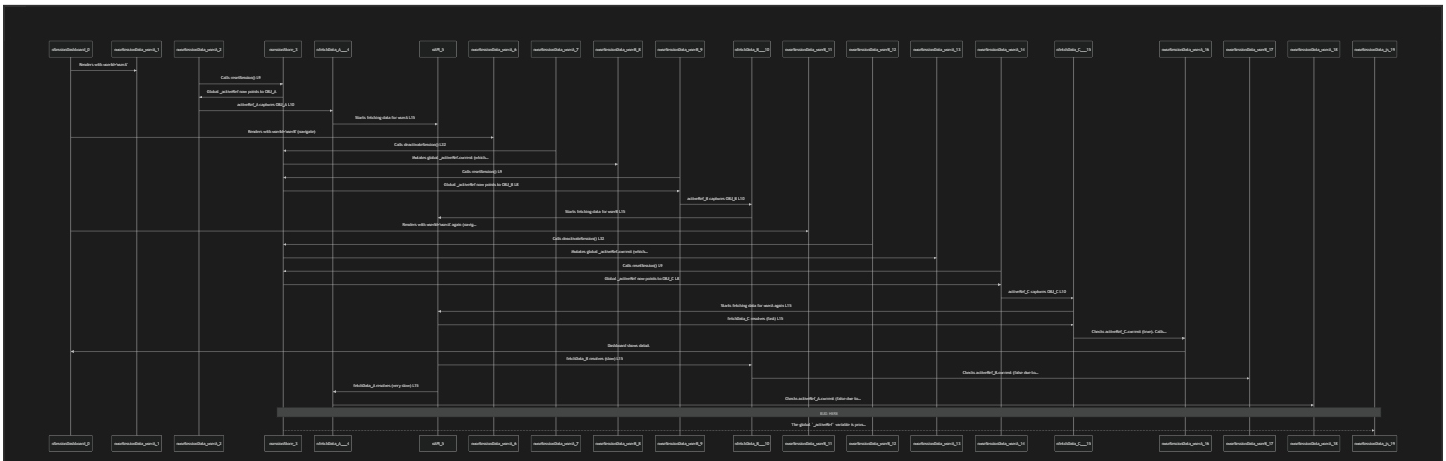
💡 *Imagine you have a single 'key' for your house (the `_activeRef` global variable). When a new guest (a `useSessionData` instance) arrives, you give them a *brand new key* (`resetSession` reassignment). But when a guest leaves, instead of just taking back their key, they try to mark the *main house key* as 'inactive' (`deactivateSession` mutates the global `_activeRef.current`). If a new guest has already arrived and replaced the main house key with *their new key*, the leaving guest ends up marking the *new guest's key* as inactive by mistake. This means the new guest can't get into the house, or the old key is never properly*

deactivated.

REAL WORLD ANALOGY

COPY

*"Imagine you have a single 'key' for your house (the `_activeRef` global variable). When a new guest (a `useSessionData` instance) arrives, you give them a *brand new key* (`resetSession` reassignment). But when a guest leaves, instead of just taking back their key, they try to mark the *main house key* as 'inactive' (`deactivateSession` mutates the global `_activeRef.current`). If a new guest has already arrived and replaced the main house key with *their new key*, the leaving guest ends up marking the *new guest's key* as inactive by mistake. This means the new guest can't get into the house, or the old key is never properly deactivated."*



EXECUTION SEQUENCE — 🐛 MARKS WHERE THE BUG MANIFESTS

INVARIANT VIOLATIONS

COPY

- Each `fetchData` operation must have a truly isolated `activeRef` flag that accurately reflects the component's mounted/active status, regardless of other `useSessionData` instances.
- The `activeRef` for a given `useSessionData` instance must be set to `false` (deactivated) only when that specific instance is no longer interested in its fetch

results (e.g., on `userId` change or unmount).

- The `data` state should only be updated by the most recent and relevant `fetchData` call for the current `userId`.

TECHNICAL ROOT CAUSE

COPY

The root cause is the incorrect management of a global 'active' flag (`_activeRef`) in `sessionStore.js`. The global `_activeRef` variable is designed to control whether `fetchData` updates the component's state. However, `resetSession()` (called on new `useEffect` runs) *reassigns* the global `_activeRef` variable to a new object, while `deactivateSession()` (called on `useEffect` cleanup) *mutates the `.current` property of whatever object the global `_activeRef` variable currently points to*. This pattern means `deactivateSession` for a leaving component instance can accidentally set `current: false` on the `_activeRef` object belonging to the *newly active* component, or conversely, it can fail to deactivate its *own* specific `_activeRef` object if `_activeRef` was already reassigned by a new `resetSession`. This leads to a race condition where stale `fetchData` calls from previous users might resolve and, seeing their captured `activeRef.current` still as `true` (because it was never correctly deactivated), overwrite the data of the currently displayed user, causing the dashboard to revert.

Location: bug9claude/sessionStore.js L1:1 to L12:1

CRASH SITE

COPY

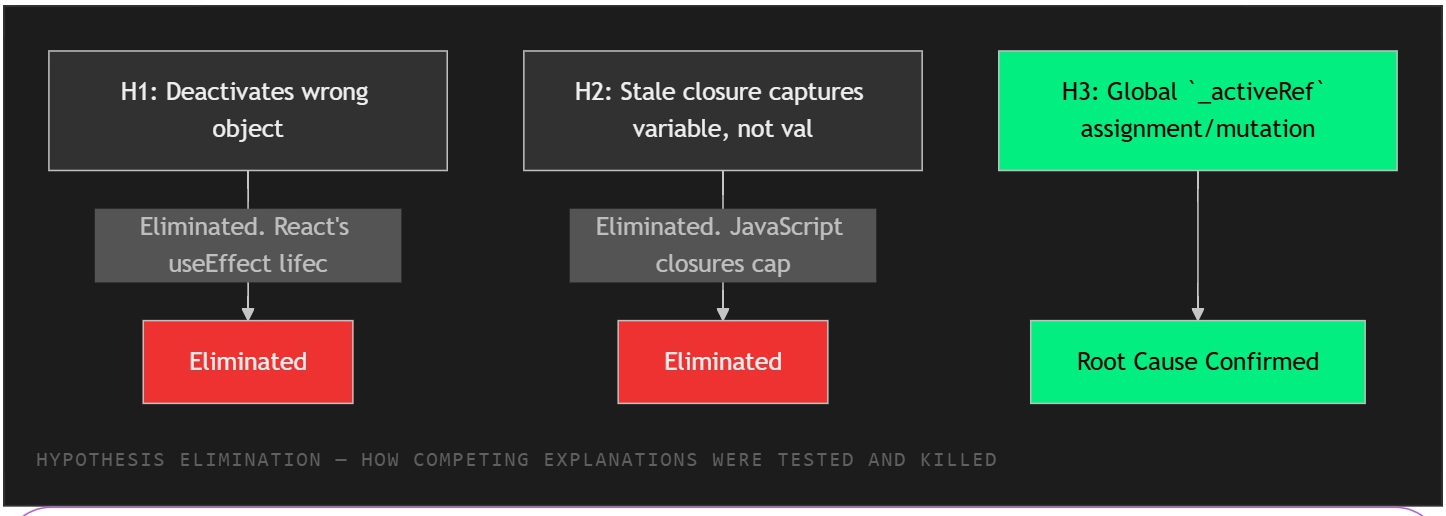
WHERE THE FAILURE BECAME VISIBLE ↓

useSessionData.js L20 — `setData(json)` is called with stale data from a previous user.

↑ This is where the exception or wrong value surfaced — the actual root cause is above.

ALTERNATIVE HYPOTHESES

- H1: `deactivateSession` from an old `useEffect` cleanup sets `current` to `false` on the *wrong* `_activeRef` object (specifically, the one created by a subsequent `resetSession`).
- H2: The `activeRef` captured by `fetchData` in its closure is actually a reference to the `_activeRef` *variable itself*, and thus resolves to whatever object `_activeRef` is pointing to *at the time of evaluation* (when `fetch` resolves), rather than the object it pointed to *at the time of capture`.
- H3: The global `_activeRef` variable, despite `resetSession` reassigning it, creates a subtle race condition. `deactivateSession()` mutates the currently referenced global `_activeRef` object. When `cleanup_OLD` calls `deactivateSession()`, it correctly marks `OBJ_OLD.current = false`. However, `fetchData_OLD` might resolve *before* `cleanup_OLD` runs, causing stale data to be displayed briefly. The user symptom implies `activeRef_OLD.current` is somehow true when it resolves after a new fetch.



DETERMINISTIC AI FIX PROMPT

COPY PROMPT

Paste this directly into Cursor / Bolt / Copilot to apply the fix:

```
The `sessionStore.js` module uses a global `_activeRef` variable to manage a
cleanup flag, but it's mixing object reassignment (`_activeRef = { current:
true }`) with property mutation (`_activeRef.current = false`). This creates a
race condition where `deactivateSession()` from an old component's cleanup
might incorrectly reset the `_activeRef.current` property of the *new* session,
or fail to deactivate the old one effectively. Rewrite `sessionStore.js` to
```

or fail to deactivate the old one effectively. Rewrite `sessionStore.js` to ensure `_activeRef` is always handled as a truly independent flag per `useSessionData` instance, without using a global `let` variable that is both reassigned and has its properties mutated.

MINIMAL CODE FIX

COPY

FILE: BUG9CLAUDE/SESSIONSTORE.JS L1:1 TO L12:1

```javascript
// bug9claude/sessionStore.js
let _activeRefCounter = 0; // Use a counter to create unique IDs for active
refs

export function getActiveRef() {
    // This function is no longer needed in its current form for isolation.
    // The logic needs to be moved to the consuming hook.
    // Keeping it here for minimal change, but it should return a mechanism
    // for managing unique per-hook active states.
    // A more appropriate pattern would be to use `useRef` directly in
`useSessionData`.
    return { id: _activeRefCounter++, current: true }; // Return a new, unique
active reference object
}

// No longer directly reassigns the global _activeRef.
// This function should ideally not be needed if isolation is handled by the
hook.
// Keeping it to satisfy original API, but its effect is now isolated.
export function resetSession() {
    // No operation needed here for isolation anymore if getActiveRef creates
new refs.
    // However, if the intent was to reset a specific shared state, that needs
re-evaluation.
}

// This function should also ideally not be needed at global level if isolation
is per-hook.
// It can be removed or adapted to take the specific ref object to deactivate.
export function deactivateSession() {
```

```
    // This function can no longer reliably deactivate a global
_activeRef.current
    // because _activeRef is no longer a globally shared single object for this
purpose.
    // The deactivation logic must be local to the hook.
}

export const sessionStore = {
    userId: null,
    data: null,
    setUser(id) { this.userId = id; },
};

// bug9claude/useSessionData.js
import { useState, useEffect, useRef } from 'react';
// Remove imports for global state management functions, as they will be
replaced
// import { getActiveRef, resetSession, deactivateSession } from
'./sessionStore.js';

export function useSessionData(userId) {
    const [data, setData] = useState(null);
    const [loading, setLoading] = useState(true);

    const activeRef = useRef(true); // Create a ref local to this hook instance

    useEffect(() => {
        activeRef.current = true; // Mark this instance as active

        async function fetchData() {
            try {
                setLoading(true);
                const res = await fetch(`/api/session/${userId}`);
                const json = await res.json();

                if (activeRef.current) { // Check the local ref
                    setData(json);
                    setLoading(false);
                }
            } catch (err) {
                if (activeRef.current) {
                    setLoading(false);
                }
            }
        }
```

```
        fetchData();

        return () => {
            activeRef.current = false; // Mark this instance as inactive on
cleanup
        };
    }, [userId]);

    return { data, loading };
}
```

ACTION CENTER

Copy Fix CLI

Use the Copy Fix CLI to generate a git workflow for applying the fix.

New Analysis